

Incident review: checkout-service

8 September 2026 • Payments • circulated 11 September

Duration	03:07 – 04:19 UTC (1 h 12 min), of which 41 minutes were full outage
Impact	An estimated 18 400 checkouts failed. 2 100 customers saw a card charged with no order created.
Trigger	Release <code>2026.09.08-1</code> , which changed the connection-pool size from 40 to 8
Detection	Automatic. The error-rate alert fired 90 seconds after the deploy finished.
Resolution	Rollback to <code>2026.09.05-3</code>

What happened

A release went out at 03:05 containing an unrelated change to the checkout timeout, and — accidentally, in the same commit — a change to the database connection pool. The pool dropped from 40 connections to 8. Under the overnight load that is enough; under the 06:00 European morning it would not have been, but we never got that far.

Requests began queueing for a connection immediately. Because the queue is unbounded, they did not fail fast: they waited, held a worker thread, and timed out at 30 seconds. Within four minutes every worker in the fleet was waiting on the pool and the service stopped answering anything at all, including its own health check.

The health check failing is the part worth dwelling on. It is what turned a degraded service into an outage, because the load balancer removed every instance at once and there was nothing left to serve the requests that would have succeeded.

The on-call engineer was paged at 03:07, acknowledged at 03:09, and rolled back at 03:58. The rollback itself took ninety seconds and worked first time.

Timeline

Time (UTC)	Event
03:05	Release <code>2026.09.08-1</code> begins rolling out.
03:06	First instance takes traffic. Connection waits appear in the logs; nobody is looking at the logs.
03:07	Error-rate alert fires. Page delivered.
03:09	On-call acknowledges in <code>#payments-oncall</code> .
03:11	Every instance is failing its health check. The service is fully out.
03:14	On-call checks the deploy history and sees the release. Begins reading the diff.
03:31	Database dashboard checked; connection count is low, not high. This is read as “the database is fine”.
03:44	Second engineer joins. Asks why the connection count is <i>low</i> during an outage.
03:56	The pool change is found in the diff.
03:58	Rollback started.
04:00	Old version taking traffic. Error rate falling.
04:19	Error rate at baseline. Incident closed.

Why it took fifty-one minutes

The rollback was the right action and it was available from the first minute. Three things stood between the page and the decision.

The runbook says to roll back first, and we did not

Our own runbook opens with “*rolling back is cheap and almost never the wrong call — do it first and diagnose afterwards.*” The on-call engineer instead read the diff to find out *what* to roll back, which is a reasonable instinct and the exact instinct the runbook exists to overrule. Reading a diff at 03:14 costs forty minutes; rolling back costs ninety seconds and loses nothing.

A low number looked like good news

The database dashboard showed a connection count of 8 where it normally shows 40. Every graph on that page is drawn so that *up* is bad, so a number that had fallen through the floor read as healthy. It was in fact the single clearest statement of the cause on any screen we have.

The change was invisible in review

The pool size lives in a configuration file that the release tooling rewrites, so the diff that a reviewer sees does not contain it. The value was changed by editing a template three weeks earlier; the release that picked it up was unrelated and its description said “*raise checkout timeout to 45 s*”. Nobody misread the change. Nobody saw it.

What we are changing

1. **Bound the connection queue.** A request that cannot get a connection within two seconds now fails rather than waiting. A service returning errors is still a service; one holding every thread is not. *Owner: Payments. Done.*
2. **Take the pool out of the health check’s path.** The check answers on a dedicated thread and reports the pool as a separate, non-fatal signal, so a struggling service degrades instead of disappearing. *Owner: Payments. In review.*
3. **Show configuration changes in the release description.** The tooling knows exactly which rendered values differ from the previous release; it simply never printed them. *Owner: Delivery. Planned.*
4. **Alert on the connection count falling, not only rising.** *Owner: Platform. Planned.*

We are deliberately *not* adding a rule that configuration changes need a second reviewer. The change was not missed for lack of eyes; it was missed because it was not on the page anyone was looking at.

What went well

- Detection was automatic and fast: 90 seconds from the deploy finishing to the page.
- The second engineer joined without being asked, and the question they asked is what ended the incident.
- The rollback worked first time, in the time it claims to take.
- Nobody tried to fix forward.

Customer impact and follow-up

2 100 customers were charged without an order being created. All were refunded automatically within six hours, and all were emailed. No manual claims have been received.

The remaining 16 300 failed checkouts saw an error page and were not charged. We have no way to tell how many came back.

Action items

Item	Owner	Due	State
Bound the connection queue at two seconds	Payments	9 Sep	Done
Health check off the pool's thread	Payments	18 Sep	In review
Print changed configuration in the release description	Delivery	30 Sep	Planned
Alert on the connection count falling	Platform	30 Sep	Planned
Re-read this document at the next on-call handover	Payments	22 Sep	Planned

This document is sample content for Multirepo Reader. The service, the release numbers and the people are invented; the failure mode is not.